

基本内容

- 在平摊分析中，执行一系列数据结构操作所需要的时间是通过执行的所有操作求平均而得出的。平摊分析可以用来证明在一系列操作中，通过对所有操作求平均之后，即使其中单一的操作具有较大的代价，平均代价还是很小的。
- 平摊分析不牵涉到概率，平摊分析保证在最坏情况下，每个操作具有的平均性能。

一. 聚集分析

聚集法：通过总运行时间求平均得到平摊时间，不需要对操作序列的概率分布做假设。

1. 栈上的三种操作

- PUSH(S,x)**: 将x压入S
 - 运行时间O(1)
- POP(S)**: 弹出栈顶
 - 运行时间O(1)
- MULTIPOP(S,k)**: 弹出栈顶k个对象 — $T(n) = O(\min\{k, s\})$

```
MULTIPOP(S, k)
1 while not STACK-EMPTY(S) and k ≠ 0
2   do POP(S)
3   k → k - 1
```

n个栈操作的平摊时间

- 设有n个栈操作 (PUSH、POP、MULTIPOP) 的序列，作用于初始为空的栈S。
- 总的运行时间的界是什么？
 - 每个操作都可能是MULTIPOP
 - 每个MULTIPOP的运行时间是 $O(\min(k, s)) = O(n)$ 并不紧
 - 总的运行时间的上界为 $O(n^2)$ 因为MULTIPOP能POP出
 - 这是一个紧的上界吗？ $\times$ $W(n) = O(n^2)$ 对象个数受PUSH操作数限制

总运行时间: $O(n)$

- 只有PUSH操作增加栈S中的对象个数
- 所有POP和MULTIPOP弹出的对象数不会弹出多于PUSH入栈的对象数
- 故总的运行时间为 $O(n)$
 - 所有PUSH入栈的对象数为 $O(n)$
 - 所有POP和MULTIPOP弹出的对象数也为 $O(n)$

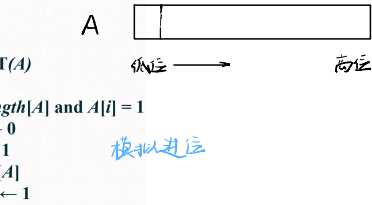
平摊时间: $O(n)/n = O(1)$

2. 二进制计数器

- 计数器 $A[0..k-1]$ 表示为k位二进制位的数组 $x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$
- 操作INCREMENT实现计数器加一
 - 运行时间 $O(k)$
 - n个INCREMENT操作序列的运行时间 $O(nk)$ (紧吗?) $\times$

不是每个INCREMENT操作都真的需要遍历A, 模拟进位

- n个INCREMENT操作序列的运行时间应为: $O(n)$
- 观察INCREMENT操作序列
 - 每次操作, $A[0]$ 都反转;
 - 每两次操作, $A[1]$ 反转;
 - 每 2^i 次操作, $A[i]$ 反转;
- 于是, 总反转次数为 $\sum_{i=0}^{\lfloor \log(n) \rfloor} \left\lceil \frac{n}{2^i} \right\rceil < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$
- 总时间 $O(n)$
- 每个操作平摊时间 $O(n)/n = O(1)$ 第i位的翻转次数 (n次操作后)



Counter value	A[16]	A[15]	A[14]	A[13]	A[12]	A[11]	A[10]	A[9]	A[8]	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	3	3
3	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	4	4
4	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	7	7
5	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	8	8
6	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	10	10
7	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	11	11
8	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	15	15
9	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	16	16
10	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	18	18
11	0	0	0	0	0	1	0	1	1	0	0	0	0	0	0	0	19	19
12	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	22	22
13	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	23	23
14	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	25	25
15	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0	26	26
16	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	31	31

二. 记账法

- 对不同的操作赋予不同的费用, 某些操作的费用比它们的实际代价或多或少。
- 我们对一个操作的收费的数量称为平摊代价。
 - 当一个操作的平摊代价超过了它的实际代价时, 两者的差值就被当作存款, 并赋予数据结构中的一些特定对象, 可以用来补偿那些平摊代价低于其实际代价的操作。
 - 记账法与聚集分析的区别: 聚集分析中的所有操作都具有相同的平摊代价。
- 数据结构中存储的总存款等于总的平摊代价和总的实际代价之差。注意: 总存款不能是负的。

1. 栈上的三种操作

- | 操作 | 实际代价 | 平摊代价 |
|----------|--------------|------|
| PUSH | 1 | 2 |
| POP | 1 | 0 |
| MULTIPOP | $\min(k, s)$ | 0 |
- 对PUSH操作多收费, 多出来的1元钱放在入栈的对象上, 待POP和MULTIPOP把该对象弹出栈时, 恰好每个对象上的1元钱抵消了操作的实际代价
 - 因为栈S中对象数不可能为负, 即存款不会小于0
 - 保证n个操作平摊时间之和是n个操作实际时间的上界

2. 二进制计数器

- 两个问题
 - 如何对INCREMENT收费? (平摊代价)
 - 收的费用放在哪里? (如何平摊)
- 注意到, 每次INCREMENT只会把一个0反转为1, 但可能把多个1反转为0。
 - INCREMENT的实际代价为反转的次数
 - INCREMENT平摊代价为2, 用于把0反转为1, 并把剩余的1元存放在反转成1的位上。
 - 把1反转成0的实际代价由存放在1上的1元钱支付
- $A[1..k-1]$ 数组中, 1的个数 (=存款) 不可能为负

练习题

- 假设我们希望不仅能使一个计数器增值, 也能使之复位至零。请说明如何将计数器实现为一个位数组, 使得对一个初始为零的计数器, 任一包含n个INCREMENT和RESET操作的序列的时间为 $O(n)$

INCREMENT: 平摊代价2
1 用掉了; 1 放在反转为1的位上
RESET: 平摊代价0

三. 势能法

- 不是将已预付的费用作为存在数据结构**特定对象**中存款来表示, 而是表示成一种“势能”或“势”, 它在需要时可以释放出来, 以支付后面操作的额外开销。
- 势是与**整个数据结构**而不是其中的个别对象发生联系的。(区别于记账法)
- 对一个初始数据结构 D_0 执行 n 个操作。对每个 i , 设 c_i 为每个操作的**实际代价**, D_i 为对数据结构 D_{i-1} 执行第 i 个操作的结果。
- 势函数 Φ 将每个数据结构 D_i 映射为一个实数 $\Phi(D_i)$, 即与 D_i 相联系的势。
- 第 i 个操作的平摊代价 a_i 根据势函数 Φ 定义为
$$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$
- 即每个操作的平摊代价为其实际代价 c_i 加上**由于该操作所增加的势**。 n 个操作的总的平摊代价为
$$\sum a_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$$

整体分析

- 如果势函数 Φ 使得对所有 n , 有 $\Phi(D_n) \geq \Phi(D_0)$, 那么总平摊代价 $\sum a_i$ 就是总实际代价 $\sum c_i$ 的一个上界。
通常为了方便起见会定义 $\Phi(D_0) = 0$ (不是必须的)
- 正的势差存储势
 - 如果第 i 个操作的势差 $\Phi(D_i) - \Phi(D_{i-1})$ 是正的, 则平摊代价 a_i 表示对第 i 个操作多收了费, 同时数据结构的势也随之增加了。
- 负的势差不足收费
 - 如果势差是负值, 则平摊代价就表示对第 i 个操作的补足收费, 这是通过减少势来支付该操作的实际代价。
- 平摊代价依赖于所选择的势函数 Φ 。
 - 不同的势函数可能会产生不同的平摊代价, 但它们都是实际代价的上界。最佳势函数的选择取决于所需的时间界。

1. 栈上的三种操作

- 定义势函数为**栈中对象的个数**
 - 开始时要处理是空栈 D_0 , 所以 $\Phi(D_0) = 0$
 - 栈中的对象数始终非负, 所以 $\Phi(D_i) \geq 0 = \Phi(D_0)$
 - 以 Φ 表示的 n 个操作的平摊代价的总和, 就是总的实际代价的一个上界 $A = C + \Phi(D_n) - 0$
- 对于作用于一个包含 s 个对象的栈上的第 i 个操作
- 如果PUSH, 则
 - 势差为 $\Phi(D_i) - \Phi(D_{i-1}) = (s+1) - s = 1$
 - 平摊代价为 $a_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 1 = 2$
- 如果是MULTIPOP(S, k)
 - 该操作的实际代价为 $c_i = \min(s, k)$
 - 势差为 $\Phi(D_i) - \Phi(D_{i-1}) = -\min(s, k)$
 - 平摊代价为 $a_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = \min(s, k) - \min(s, k) = 0$
- 类似的, 如果是POP, 平摊代价也为0
- 三种栈操作**每种的平摊代价都是 $O(1)$** , 这样包含 n 个操作的序列的总平摊代价就是 $O(n)$ 。已经证明 n 个操作的平摊代价的总和是总的实际代价的一个上界。所以 n 个操作的最坏情况代价为 $O(n)$

2. 二进制计数器

- 定义势函数为数组 $A[0...k-1]$ 中**1的个数**
计算INCREMENT的平摊代价
 - 设第 i 次操作把 t_i 个1反转为0, 则实际代价为 $t_i + 1$
 - 设第 i 次操作后数组中1的个数为 b_i
 - 如果 $b_i = 0$, 则第 i 次操作反转了 k 个1, 有 $b_{i-1} = t_i = k$
 - 如果 $b_i > 0$, 则 $b_i = b_{i-1} - t_i + 1$ (多进位的一个1)
 - 两种情形下都有: $b_i \leq b_{i-1} - t_i + 1$
- 势差为 $\Phi(D_i) - \Phi(D_{i-1}) \leq (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i$
- 平摊开销为 $a_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) \leq (t_i + 1) + (1 - t_i) = 2$

overflow

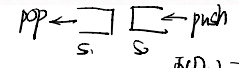
该势函数并未要求 $\Phi(D_0) = 0$, 因此对初值非0的计数器也成立。

- 设计计数器中初始有 b_0 个1, n 次INCREMENT操作后有 b_n 个1。 $(0 \leq b_0, b_n \leq k, k \text{ 为计数器位数})$
对所有的 i , $a_i \leq 2$ (见前)
 $\Phi(D_n) = b_n, \Phi(D_0) = b_0$
 n 次INCREMENT操作总代价
$$\sum c_i = \sum a_i - \Phi(D_n) + \Phi(D_0) \leq \sum 2 - b_n + b_0 = 2n - b_n + b_0 = O(n)$$

只要 $k \leq O(n)$, 则总代价就为 $O(n)$
- 这里并没有要求 $\Phi(D_n) \geq \Phi(D_0)$, 也没有要求 $\Phi(D_0) = 0$ (?)

练习题

- 考虑普通二叉最小堆上最坏运行时间为 $O(\log n)$ 的操作INSERT和EXTRACT-MIN。请给出势函数 Φ , 使得INSERT操作的平摊代价为 $O(\log n)$, EXTRACT-MIN的平摊代价为 $O(1)$ 。
- 说明如何用两个普通的栈来实现一个队列, 使得每个ENQUEUE和DEQUEUE操作的平摊代价都为 $O(1)$ 。

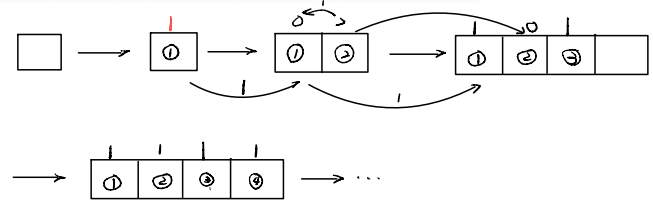


例: 动态表上的平摊分析

- 在有些应用中, 在开始的时候无法预知在表中要存储多少个对象。这就希望能根据对象的多少调整**所需要的存储空间** (多退少补)
- 表的动态扩张和收缩——动态表
 - 用平摊分析证明插入、删除操作的平摊代价是 $O(1)$
 - 动态表的具体结构可以是堆、栈、散列表等等
 - 假设我们用**数组来存储动态表** (不是链表)
 - 通过插入扩张和删除搜索
 - 保证未使用空间总是**不多于**整个分配空间的一定比例
- 先看只有插入操作的情形, 再拓展也有删除操作

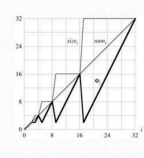
```
Insert(T, x)
1 if size[T] == 0
2 then 给table[T]分配一个槽的空间
3 size[T] ← 1
4 if num[T] == size[T]
5 then 分配一个有2*size[T]个槽的空间的新表
6 将table[T]中所有的项插入到新表中
7 释放table[T]
8 table[T]指向新表的存储块地址
9 size[T] ← 2*size[T]
10 将x插入table[T]
11 num[T] ← num[T] + 1
```

- 简单分析, 一次插入操作的代价为 $O(n)$ $W_{in} = O(n)$
 - 于是 n 次插入操作总代价为 $O(n^2)$
- 聚集分析, 一次插入操作的代价
 - 当 $i-1$ 是2的幂时: $c_i = i$ (重新分配)
 - 其他时候: $c_i = 1$ (直接插入)
 - 总代价 $\sum_{i=0}^n c_i \leq n + \sum_{i=0}^{\log n} 2^i \leq n + 2n = 3n$
 - 平摊代价为 $3n/n = 3$ (团转总行)
- 记账法, 插入收费3, 其中2元钱分别赋给表中最后插入的对象和一个已经没有存款的对象
当需要扩张表时, 每个对象被复制一次, 恰好用完存款



势函数

- 势函数: $\Phi(T) = 2 \cdot \text{num}[T] - \text{size}[T] \geq 0$
- 当第 i 个操作不触发表扩张, 则
$$a_i = c_i + \Phi_i - \Phi_{i-1} = 1 + (2 \cdot \text{num}_i - \text{size}_i) - (2 \cdot \text{num}_{i-1} - \text{size}_{i-1}) = 1 + (2 \cdot \text{num}_i - \text{size}_i) - (2(\text{num}_i - 1) - \text{size}_i) = 3$$
- 当第 i 个操作触发表扩张, 则
$$a_i = c_i + \Phi_i - \Phi_{i-1} = \text{num}_i + (2 \cdot \text{num}_i - \text{size}_i) - (2 \cdot \text{num}_{i-1} - \text{size}_{i-1}) = \text{num}_i + (2 \cdot \text{num}_i - 2(\text{num}_i - 1) - (2(\text{num}_i - 1) - \text{size}_{i-1})) = \text{num}_i + 2 - (\text{num}_i - 1) = 3$$



表扩张和收缩时的平摊代价

- 插入/删除操作：满时扩张、小于1/4时收缩
 - 势函数
 - $\Phi(T) = \max(2\text{num}[T] - \text{size}[T], \text{size}[T]/2 - \text{num}[T])$
 - 插入操作的平摊代价都小于3
 - 根据是否引起扩张来分别计算
 - 删除操作的平摊代价都小于2
 - 根据是否引起收缩来分别计算
 - 总平摊代价为 $O(n)$
-
- 插入/删除操作：满时扩张、小于1/3时收缩，势函数？